

MINUTA DE REQUERIMIENTOS

Sistema de Gestión de Reparaciones de PC "TecnoService"

Módulo Programador | Hito 2 - Evidencia 5 | Equipo Bit&Fix (Comisión B1)

1. Objetivo de este documento

Esta minuta complementa el análisis y el diagrama entidad-relación ya presentados en la Evidencia 4, respondiendo puntualmente a situaciones operativas del taller que el análisis inicial no dejaba resueltas por escrito. Cada definición adoptada acá queda reflejada en el modelo de datos, el script DDL y el prototipo Tkinter.

2. Usuarios del sistema

Se identifican tres roles de usuario, cada uno mapeado a un rol de PostgreSQL con permisos diferenciados (ver `script_tecnoservice.sql`):

Rol	Qué necesita ver / hacer
Administrador	Alta de técnicos, catálogo de repuestos, ve todas las órdenes, aprueba presupuestos, reportes.
Técnico	Ve solo las órdenes asignadas a su legajo, carga diagnóstico, asocia repuestos, actualiza estado.
Recepcionista	Registra cliente y equipo, redacta la falla, asigna técnico, genera la orden inicial.

3. Situaciones aclaradas

3.1 ¿Qué pasa si un equipo vuelve a ingresar por otra falla?

Pregunta: Un cliente ya trajo su notebook una vez. Semanas después vuelve con el mismo equipo, pero por un problema distinto. ¿Se crea un registro de equipo nuevo o se reutiliza el que ya existía?

Definición adoptada: Se reutiliza el mismo equipo. La tabla EQUIPO representa el hardware en sí (marca, modelo, número de serie), no una visita puntual al taller. Cada vez que el equipo ingresa —sea la primera vez o la quinta— se crea una fila nueva en REPARACION vinculada al mismo `id_equipo`, con su propia `falla_reportada`, diagnóstico y estado independientes de las visitas anteriores.

Cómo se implementa: Cardinalidad EQUIPO (1) -- <registra> -- (N) REPARACION en el modelo E-R. En la interfaz, la pestaña Ingresos siempre genera una reparación nueva; si el DNI del cliente ya existe, se reutiliza el cliente, y si se vuelve a cargar el mismo equipo, queda como una fila de EQUIPO adicional asociada al mismo cliente (ver limitación en el punto 4).

3.2 ¿Qué sucede si una reparación necesita un repuesto y no hay stock suficiente?

Pregunta: El técnico quiere asociar 2 unidades de un repuesto a una orden, pero en el depósito solo queda 1. ¿La aplicación lo permite igual?

Definición adoptada: No. El sistema debe impedir la operación y avisar al usuario cuánto stock hay disponible, sin dejar nunca el stock en un valor negativo. Recién cuando la cantidad solicitada es cubierta por el stock disponible, se descuenta automáticamente del inventario.

Cómo se implementa: A nivel de base de datos: trigger `trg_validar_stock` (función `fn_validar_y_descontar_stock`) en BEFORE INSERT de `detalle_reparacion`, que aborta la transacción con RAISE EXCEPTION si el stock es insuficiente, y descuenta el stock si la operación es válida. A nivel de prototipo: `acceso_datos.usar_repuesto_en_reparacion()` replica la misma regla y lanza `StockInsuficienteError`, que la interfaz muestra como un mensaje de error al usuario.

3.3 ¿Cómo se registra si el cliente acepta o rechaza el presupuesto?

Pregunta: Después de diagnosticar el equipo, se le informa un presupuesto al cliente. ¿Dónde queda registrado si el cliente lo aceptó, lo rechazó, o todavía no respondió? ¿Se puede empezar a reparar sin que el cliente haya aceptado?

Definición adoptada: Se agrega el atributo presupuesto_aprobado a REPARACION, con tres valores posibles: Pendiente (default, recién presupuestada), Aceptado o Rechazado. No se puede avanzar el estado de la orden a 'En Reparación', 'Terminado' ni 'Entregado' mientras el presupuesto no esté en 'Aceptado' — esto evita reparar (y gastar repuestos) sobre un presupuesto que el cliente todavía no aprobó.

Cómo se implementa: A nivel de base de datos: *CONSTRAINT chk_presupuesto_aprobado_para_reparar* en la tabla REPARACION. A nivel de prototipo: *acceso_datos.puede_avanzar_estado()* aplica la misma regla antes de aceptar el cambio de estado, y la interfaz muestra el motivo del rechazo si corresponde.

4. Limitación conocida (a resolver)

La minuta identificó que, en la pestaña Ingresos, cargar el mismo equipo dos veces (por ejemplo buscándolo por número de serie) todavía no está resuelto en el prototipo: cada ingreso crea una fila de EQUIPO nueva en lugar de ofrecer reutilizar una ya existente. El modelo de datos ya lo permite (por eso el punto 3.1 se puede responder), pero la búsqueda de equipos existentes por número de serie antes de crear uno nuevo se implementará al integrar la base real próximamente, donde sí tiene sentido hacer esa consulta contra la tabla EQUIPO.